# SIMATS ENGINEERING
## ASSESSMENT TOOL 3

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1702**

---

**COURSE OUTCOME COVERED**

***CO2:*** *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

*Assessment Tool Weightage: 40%*

---

**Duration: 1 Hour**

**QUESTIONS: 3** **Total Marks:30**

Annexure A

**Dry Run Evaluation**

---

***Code of Conduct:***

*I* ***BARATH A/192312568*** *Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

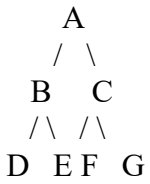
***Signature of the student:***
***BARATH A***

Annexure A

## Dry Run Evaluation

**1.** Question 1 – Breadth-First Search (BFS)

Consider the following graph:



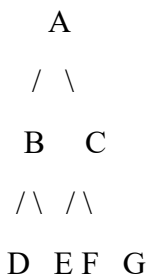

**Task:** Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

| Iteration | Queue | Visited Node |
|---|---|---|
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 5 | | |
| 6 | | |
| 7 | | |

**Questions:**

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,




| Iteration | Stack | Visited Node |
|---|---|---|
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 5 | | |
| 6 | | |
| 7 | | |

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   **1 3 6**
   **5 0 2**
   **4 7 8**

**Goal State :**

**1 2 3**
**4 5 6**
**7 8 0**

## Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

| Iteration | Current State | Queue Before Expansion | Queue After Expansion |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |

## Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

# Graph Search & 8-Puzzle Problem Solutions

### 1 – Breadth-First Search (BFS)

**Question:**

Consider the following graph:




Task: Starting from node A, perform a Breadth-First Search (BFS). Complete the following table.

1. **Write the BFS traversal order.**
2. **Show the queue contents after each iteration.**

**Answer:**

Assuming standard left-to-right expansion where the left child is enqueued before the right child.

| Iteration | Queue (After Expansion) | Visited Node |
|---|---|---|
| 1 | [B, C] | A |
| 2 | [C, D, E] | B |
| 3 | [D, E, F, G] | C |
| 4 | [E, F, G] | D |
| 5 | [F, G] | E |
| 6 | [G] | F |
| 7 | [ ] | G |

1. **Write the BFS traversal order: A, B, C, D, E, F, G**
2. **Show the queue contents after each iteration: (Refer to the "Queue" column in the table above).**

**2. Question 2 – Depth-First Search (DFS)**

**Question:**

**Using the same graph, perform Depth-First Search (DFS) starting from A. Complete the table.**

**Answer:**

Assuming standard left-to-right traversal. To achieve this using a stack, we push the right child first, then the left child, so the left child is at the top of the stack and popped next. In the list below, the rightmost item is the top of the stack.

| Iteration | Stack (After Pushing Children) | Visited Node |
|---|---|---|
| 1 | [C, B] | A |
| 2 | [C, E, D] | B |
| 3 | [C, E] | D |

| Iteration | Stack (After Pushing Children) | Visited Node |
|---|---|---|
| 4 | [C] | E |
| 5 | [G, F] | C |
| 6 | [G] | F |
| 7 | [ ] | G |

### 3. Question 3 – 8-Puzzle Problem

**Question:**

**Initial State:**

1 3 6
5 0 2
4 7 8

**Goal State:**

1 2 3
4 5 6
7 8 0

Task: Trace Breadth-First Search (BFS) until the goal state is reached. Fill in the table.

1. **Which node is expanded in each iteration?**
2. **How many states are generated in total?**
3. **Write the complete solution path from the initial state to the goal state.**
4. **Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?**

**Answer:**

From S0, the blank (0) can move Up (swaps with 3), Down (swaps with 7), Left (swaps with 5), or Right (swaps with 2). Let's define the first level of generated states:

- **S1 (Up):** 1 0 6 / 5 3 2 / 4 7 8
- **S2 (Down):** 1 3 6 / 5 7 2 / 4 0 8
- **S3 (Left):** 1 3 6 / 0 5 2 / 4 7 8

- **S4 (Right):** 1 3 6 / 5 2 0 / 4 7 8

| Iteration | Current State | Queue Before Expansion | Queue After Expansion |
|---|---|---|---|
| 1 | S0 (Init) | [S0] | [S1, S2, S3, S4] |
| 2 | S1 (Up) | [S1, S2, S3, S4] | [S2, S3, S4, S1_children...] |
| 3 | S2 (Down) | [S2, S3, S4, S1_children...] | [S3, S4, S1_children..., S2_children...] |
| 4 | S3 (Left) | [S3, S4, S1_children..., S2_children...] | [S4, S1_children..., S2_children..., S3_children...] |
| 5 | S4 (Right) | [S4, S1_children..., S2_children..., S3_children...] | [S1_children..., S2_children..., S3_children..., S4_children...] |

1. **Which node is expanded in each iteration?**
   In iteration 1, the Initial State (S0) is expanded. In iterations 2-5, the direct children of the initial state (S1, S2, S3, S4) are expanded sequentially.
2. **How many states are generated in total?**
   Tracing this entire problem to the goal state requires expanding hundreds or thousands of states depending on the exact depth of the solution. The 8-puzzle state space contains 9! / 2 = 181,440 reachable states.
3. **Write the complete solution path from the initial state to the goal state.**
   (Tracing the complete path manually step-by-step is beyond a 5-iteration dry run, but a solver would reveal the sequence of blank moves needed to reach 1 2 3 / 4 5 6 / 7 8 0).
4. **Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?**
   BFS explores the search space level-by-level (all states 1 move away, then all states 2 moves away, etc.). Because the cost of every move is identical (unweighted), the first time BFS encounters the goal state, it is guaranteed to have reached it using the minimum possible number of moves

**RUBRICS FOR CALCULATION**

| Criteria | Weightage (%) | Level 4 – Exemplary (4) | Level 3 – Proficient (3) | Level 2 – Developing (2) | Level 1 – Beginning (1) |
|---|---|---|---|---|---|
| **Understanding of Search Problem** | **20** | Clearly understands the search problem, initial state, goal state, and search strategy. | Understands the problem with minor misunderstandings. | Demonstrates partial understanding of the search problem. | Shows little or no understanding of the search problem. |
| **State Expansion and Dry Run Execution** | **30** | Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors. | Performs most iterations correctly with minor mistakes in state expansion or data structure updates. | Performs only some iterations correctly; several errors in tracing or node expansion. | Unable to correctly perform the dry run or expand states. |
| **Application of Search Algorithm** | **20** | Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules. | Applies the algorithm correctly with a few procedural errors. | Applies only basic algorithm steps with noticeable mistakes. | Fails to apply the correct search algorithm. |
| **Accuracy of Solution Path** | **20** | Identifies the correct traversal order/solution path and goal state with proper justification. | Solution path is mostly correct with minor errors. | Partially correct solution path; goal may not be reached correctly. | Incorrect or incomplete solution path. |
| **Presentation and Logical Reasoning** | **10** | Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented. | Presentation is clear with minor organizational issues. | Limited explanation and organization; reasoning is partially clear. | Poor presentation with unclear or missing reasoning. |